Practice Exercises

Work through these exercises after completing each chapter. Use **MySQL 8+** and the `training_db` database from MySQL Setup.

Load sample rows with `labs/training_db_seed.sql` before chapter 03. Chapter 02 still has you `CREATE player` and `employee` yourself.

Map of modules: MYSQL README.

> **Module Outcome:**
>
> - Drill each chapter against `training_db` until the syntax is in your fingers
> - Write the query first, then check — not copy from a neighbour
> - Treat this file as gym work, not a new topic

Chapter 00 — MySQL Setup

1. Install MySQL and MySQL Workbench on your machine.
2. Connect via CLI using `mysql -u root -p`.
3. Create database `training_db` and run `USE training_db;`.
4. Run `DESCRIBE` on any table you create.
5. Create a practice user with `GRANT SELECT ON training_db.* TO 'trainee'@'localhost';`

Chapter 01 — Introduction to Databases

1. Explain the difference between **data**, **database**, and **DBMS** in your own words.
2. List three real-world applications that use a relational database and three that use a non-relational database.
3. Why is a relational database the most commonly used type for business applications?

Chapter 02 — Introduction to SQL (Basic Concepts)

Setup: Do **not** use the seed for this chapter. Create the `player` table yourself (`id`, `name`, `age`, `score`).

1. Write a `CREATE TABLE` statement for `player` with `id` as `PRIMARY KEY AUTO_INCREMENT`.
2. Insert three players. Let MySQL auto-assign `id` for at least one row.
3. Select only `name` and `score` for all players.
4. Update the score of the player named `'Virat'` to `75`.
5. Delete the player with `id = 2`.
6. Add a column `jersey_number` (INTEGER) to the `player` table using `ALTER TABLE`.
7. Create an `employee` table with `NOT NULL` on `name`, `UNIQUE` on `email`, and `CHECK (salary > 0)`.
8. **Challenge:** Insert a row with `salary = -100` into `employee`. What happens?

Chapter 03 — Querying with SQL

Setup: Run `labs/training_db_seed.sql`. Use the `product` table (`name`, `category`, `price`, `brand`, `rating`).

1. Get all products with `price` greater than `500`.
2. Get all products whose `name` contains `"cake"`.
3. Get all products whose `brand` has exactly 4 characters (use `_` wildcard).
4. Get all clothing products with `price` between `500` and `1000` (inclusive).
5. Get all products where `brand` is `"puma"`, `"Denim"`, or `"Apple"` using `IN`.
6. Get all products where `brand IS NULL`.
7. Get the top 3 highest-rated products using `ORDER BY` and `LIMIT`.
8. Get distinct `category` values sorted alphabetically.
9. Get products ranked 4th to 6th by `price` (descending) using `LIMIT` and `OFFSET`.
10. **Challenge:** Write a query using `NOT`, `AND`, and `OR` together: products that are either in the `"Food"` category with `rating > 4`, or in `"Gadgets"` with `price < 5000`.

Chapter 04 — Aggregations and Group By

Setup: Seed loaded. Use the `player_match_details` table (`name`, `score`, `year`).

1. Find the total runs scored by `"Ram"`.
2. Find the average score across all matches.
3. Count the total number of matches using `COUNT(*)`, `COUNT(1)`, and `COUNT(score)`. Compare results.
4. Get each player's total score with column alias `total_runs`.
5. Get the number of half-centuries (`score >= 50`) per player.
6. Get only players who have more than one half-century (use `HAVING`).
7. **Challenge:** Get the year with the highest average score.

Chapter 05 — Common Concepts

Setup: Seed loaded. Use the `movie` table (`id`, `name`, `genre`, `budget_in_cr`, `collection_in_cr`, `rating`, `release_date`). Actor IDs live in `movie_actor`.

1. Select `name` and profit (`collection_in_cr - budget_in_cr`) for all movies.
2. Get movies released in the year `2010` using `YEAR()` or `DATE_FORMAT()`.
3. Get the count of movies released per year.
4. Round `rating` to 1 decimal place using `ROUND`.
5. Find movies whose name contains `"avenger"` (case-insensitive) using `UPPER` / `LOWER`.
6. Get actor IDs who appear in both movie `6` and movie `15` using `INTERSECT`.
7. Get unique actor IDs from movie `6` or `15`, sorted descending, limited to 5 rows.
8. **Challenge:** Get each genre's average profit using `GROUP BY` and `HAVING` (only genres with average profit > 50).

Chapter 06 — Modeling Databases

1. Draw an ER diagram for a **library system** with: `Book`, `Member`, `Loan` (a member borrows many books over time; each loan is for one book).
2. Identify the relationship types (1:1, 1:N, M:N) in the e-commerce model (`customer`, `cart`, `product`, `address`).
3. Write `CREATE TABLE` for a `loan` table with a foreign key to `member` and `ON DELETE CASCADE`.

4. Why is a junction table needed for cart–product relationship?
5. **Challenge:** Add a `CHECK` constraint so `quantity` in `cart_product` must be greater than 0.

Chapter 07 — Joins

Setup: Seed loaded. Use the e-learning tables: `student`, `course`, `instructor`, `student_course`, `review`.

1. Use `NATURAL JOIN` to list courses taught by `"Alex"`.
2. Use `INNER JOIN` to get reviews for course id `15` with student names.
3. Use `LEFT JOIN` to find students who have not enrolled in any course.
4. Use a self-join on `student_course` to find pairs of students in the same course.
5. Get the highest score in each course using `LEFT JOIN` and `MAX`.
6. **Challenge:** Get instructor name, course name, and enrolled student count per course.

Chapter 08 — Views and Subqueries

Setup: Seed loaded. Use `product` and `product_order`.

1. Create a view `high_rated_products` showing products with `rating > 4.5`.
2. Query the view to list product names.
3. Get products with `price` greater than the average price (subquery in `WHERE`).
4. Get products whose `category` average price is shown alongside each product (correlated subquery in `SELECT`).
5. Use `EXISTS` to find brands that have at least one product rated above `4.5`.
6. Use `NOT EXISTS` to find products not present in any order.
7. **Challenge:** Rewrite exercise 3 using a `JOIN` instead of a subquery.

Chapter 09 — Transactions and Indexes

Setup: Seed loaded. Transfers use the `accounts` table (`Sam`, `David`). Index drills use `product`. Use the `employee` table you created in chapter 02 for the unique index.

1. Explain ACID properties with the bank transfer example in your own words.
2. Write a transaction that debits `500` from Sam and credits `500` to David using `BEGIN`, `COMMIT`.
3. What happens if you `ROLLBACK` instead of `COMMIT` after the two `UPDATE` statements?
4. Create an index on `product(brand)`.
5. Create a unique index on `employee(email)`.
6. When would you **not** want to add an index on a column?
7. **Challenge:** Write a transaction that inserts a new order and order items. Roll back if any insert fails.
8. Grant `SELECT` only on `training_db` to `'readonly'@'localhost'`, then `REVOKE` it.
9. Use `SAVEPOINT` during a transfer and `ROLLBACK TO SAVEPOINT` for partial undo.

Chapter 11 — Window Functions & CTEs

Setup: Seed loaded. Ranking drills use `product`.

1. Rank products by price within each category using `ROW_NUMBER()`.
2. Find the top-rated product per category using `ROW_NUMBER()` in a subquery.
3. Rank players by score using `RANK()` and `DENSE_RANK()` — compare outputs when scores tie. Use `player_match_details`.
4. Write a CTE to find products priced above the average.
5. Write a recursive CTE for a 3-level employee hierarchy (create a small `org_chart` table for this drill).
6. **Challenge:** Use `DENSE_RANK()` to find the 2nd highest-rated product in each category.

Chapter 12 — Real-Time Projects

Complete at least **one** full project from Real-Time Projects:

- **AcmeHR** — all 7 practice queries + challenge
- **TechMart** — all 7 practice queries + challenge
- **MedCare** or **SecureBank** — all 6 practice queries + challenge

---

Mixed / Capstone Exercises

1. **E-commerce report:** For each brand, show total products, average rating, and highest-priced product name.
2. **Player leaderboard:** Rank players by total score. Show rank, name, total score, and number of matches played.
3. **Course analytics:** List courses with fewer than 2 enrolled students (use `LEFT JOIN` + `GROUP BY` + `HAVING`).
4. **Data cleanup:** Find and list duplicate `brand` + `name` combinations in `product`.
5. **Schema design:** Design and create tables for a **movie ticket booking** system (movies, theatres, shows, bookings). Include PKs, FKs, and at least one `CHECK` constraint.

Answer Hints

▶ Chapter 02 — Exercise 1 hint
▶ Chapter 03 — Exercise 6 hint
▶ Chapter 09 — Exercise 2 hint